Firmware 4.0.0 data in pyflic

What pyflic does with each column of a DFM data file, by **Generation**, and where each one surfaces. For a developer or an analyst who needs to know which output a column feeds, or why a column is `NA`.

The file formats themselves are PyMCU's to define. This document says only what pyflic reads and how; where it and PyMCU's documents disagree about the file, PyMCU's win:

What	Where
Every data-file column, its missing-value rule, when it is meaningful	`PyMCU/docs/DataFileColumns.md`
The Program file, the six Paradigm types, their settings and defaults, the `Program.txt` export	`PyMCU/docs/ProgramFormat.md`
The decision and its reasons	ADR-0016, and the addenda to ADR-0014 and ADR-0015
The plan, its decisions and where the build departed from it	v4-data-plan.md
The vocabulary (Generation, Paradigm, Program Interval, Activation, Requirement, Progress, Firmware Event, Phase, Acclimation, Hold, Role)	../CONTEXT.md

The PyMCU links assume its checkout sits beside pyflic's, on branch `v4_Dev`. For a short overview of the new columns and where they show up, see The firmware 4.0.0 data columns. For the researcher's view of the same features see the help topic *Optogenetic experiments* and Checking your data.

- Generations
- The recording folder
- Loading
- The original 27 columns
- The 63 new columns
- Program.txt
- Messages.csv
- What pyflic derives
- Outputs by file
- To confirm on the first real recording
- Not read

Generations

No file states its firmware or PyMCU version. Each DFM's concatenated files are classified once at load (`paradigm_state.detect_generation`), from `Phase1`, the one Paradigm column the firmware fills on every row:

Generation	Constant	Written by	Header	`Phase1`	`DFM.training_info`
1	`GENERATION_FLAG`	PyMCU 3.x, any older firmware	27 columns	absent	`flag`
2	`GENERATION_EMPTY`	PyMCU 4.0.0, DFM firmware before 4.0.0	90 columns	every value `NA` or blank	`none`
3	`GENERATION_PARADIGM`	PyMCU 4.0.0, DFM firmware 4.0.0	90 columns	any `A` or `M`	`phase`

`DFM.generation` carries it, and `with_params` copies it. The `version` (2 or 3) that the file name gives stays as a separate axis. The generation is written to `summary.txt` (*DFM details*), the integrity report, the report cover's **Generation** line, the `Generation` column of the light table and of `opto_light_qc.csv`, and the Project Report's member table.

The recording folder

File	pyflic
`DFM<id>_<n>.csv`	read and concatenated in natural order of `n` (a new file every 12 hours, each with its header)
`Program.txt`	parsed into the Opto Program (`opto_program.read_program`); optional; more than one stops the load
`Messages.csv`	read by `messages.read` for the integrity report and the QC Viewer; optional
`EnvMon_<n>.csv`	the Environmental Monitor (id 99); never read

DFM id 99 is the Environmental Monitor's. PyMCU writes its data as `EnvMon_<n>.csv`, which pyflic's `DFM<id>_*.csv` discovery never matches. pyflic also excludes id 99 everywhere a DFM id is discovered (`layout`, `yaml_config`, `project`), so a `DFM99_<n>.csv` is never scaffolded or loaded either, and a config naming `id: 99` is refused with *DFM 99 is the Environmental Monitor, not a DFM* (`utils.ENVIRONMENTAL_MONITOR_MESSAGE`), at load and by `pyflic lint`.

The disk cache's key (`cache._CACHE_VERSION "3"`) includes the modification times of `Program.txt` and `Messages.csv`, so either changing recomputes the member.

Loading

- **Missing values** are the literal `NA` in released PyMCU 4.0.0, and blank in development builds before 16 October 2026. pyflic accepts both. A leading run of missing `Interval` is the baseline, 0, as those builds wrote it.
- **The Paradigm columns leave `raw_df`.** `split_paradigm_columns` takes the 63 new columns off the frame the rest of pyflic copies, and on generation 3 `ParadigmState.from_frame` holds them as numpy arrays, row for row with `raw_df`, as `DFM.paradigm`. Nothing downstream reads them except through it. On generations 1 and 2 `DFM.paradigm` is `None`.
- **Repeated records are dropped.** A row whose `Index`, `Date`, `Time` and `MSec` an earlier row already has is a block PyMCU wrote twice after a failed file write; the first copy is kept, in `raw_df` and in the Paradigm state alike, and the count goes to `DFM.repeated_records` and a load note (`DFM.load_notes`). A DFM that restarted and counts its `Index` from 1 again with new times loses nothing.
- **A range filter** applies to the Paradigm state as to `raw_df` (`ParadigmState.subset`).

The original 27 columns

Unchanged in name and position since PyMCU 3.x.

Column	Generation 1	Generation 2	Generation 3	Surfaces in
`Date`, `Time`, `MSec`	the recording clock: `Seconds` and `Minutes`	same	same; PyMCU derives them from recording start + `Index` × 200 ms, so a lost record is a time gap	every time axis; integrity start and end; `data_breaks`; the repeated-record match
`Sample`	read	read	read	`data_breaks` rows; the generation-3 training end's `Sample`
`Index`	gaps are lost records	same	same, through `ParadigmState.index_gaps`	integrity report (`lost_records`, the gaps), the report's *Lost records*, the light QC's *lost records* (generation 3)
`W1`–`W12`	the signal; a value above 40000 is a well in training, its signal the value − 65536	the signal, never offset	the signal, never offset	every lick, event and feeding metric; on generations 1 and 2 also the emulated firmware trigger
`Temp`, `Humid`, `LUX`,	kept in `raw_df`	same	same	nowhere

Column	Generation 1	Generation 2	Generation 3	Surfaces in
`VoltsIn`				
`Dark`, `OptoFreq`, `OptoPW`	compared with each scheduled interval's program	same	compared with each Program Interval's Paradigm; 0 on the baseline rows	the light QC's *program mismatch*
`OptoCol1`	the light per well (`lights_df`)	same	`lights_df` is still built, but `OnMs` is the light	generations 1 and 2: the light QC's lit samples and light events, the Progressive Ratio light events and `LightOn_sec`, `OptoOn_sec*`, the light table's `LitSec` (`LitSource` = `OptoCol1`) and `LightOnsets`, the light-phase summary, the timeline's light strip
`OptoCol2`	decoded with `OptoCol1`	same	always 0	—
`Error`	bits 0x01 and 0x02 counted together as *No ACK / overflow*	0x01 *No Ack*, 0x02 *Overflow*	same as 2	the integrity report (`dfm.ERROR_BITS`), the report's error flags; bits 0x04–0x80 are labelled the same on every generation

The 63 new columns

Every one is `NA` on generation 2; pyflic reads them only on generation 3.

Column	pyflic reads it as	Used for	Surfaces in
`Interval`	`ParadigmState.program_interval`; `NA` is an unknown instruction, 0 the baseline	the windows of everything per Program Interval (`interval_runs`); the lead-in (`lead_in_rows`); unknown-instruction rows	the light QC's windows and `opto_light_intervals.csv`'s `ProgramInterval`; Program Facets; the light table's `ProgramInterval`; the timeline's band; the integrity report's unknown-instruction rows. Always exposed as `ProgramInterval`, never bare `Interval`
`Cycle`	`ParadigmState.cycle`	with `Interval`, one run of one Program Interval	`program_cycles` Facets (`C<n>`); `Cycle` in the light table and `opto_light_intervals.csv`; the timeline's band labels
`Paradigm`	`ParadigmState.paradigm`; the join key to `Program.txt`'s `Paradigms:` block (names matched ignoring case)	each row's Paradigm type and settings; a change of name resets every well's state	the light QC (*paradigm not in program*, *program mismatch*); Program Facet labels; the light table's `Paradigm` and `ParadigmType`; the integrity report's Paradigm sequence; the Progressive Ratio Test Paradigm (`test_paradigm`)

Column	pyflic reads it as	Used for	Surfaces in
`Phase1`–`12`	`ParadigmState.phase_a`: `A` true, anything else false	Acclimation runs and ends; Training (decision 11)	Progressive Ratio training end and notes; the light QC's Acclimation rule and *acclimation count mismatch*; the light table's Acclimation columns; the ledger's `Phase`; the timeline's shading; the QC Viewer's `Phase`
`Activations1`–`12`	`ParadigmState.activations`, cumulative; a missing value carries the previous total; a rise marks an Activation's onset row	light events; trigger wells without a program	the light QC's light events (`Source` = `activations`) and Explained Light; Progressive Ratio Light Events; the light table's `Activations`; `Activations` on the feeding summary; Roles without a program; *broken yoking* (an `Off` well that counts); the timeline's ticks
`Requirement1`–`12`	`ParadigmState.requirement`; `NA` off a ratio Paradigm	the Requirement an Activation met, read from the row before the rise (`requirement_met`)	the ledger's `Requirement`; `BreakRequirement`; the light table's `LastRequirementMet`; `RequirementMet` in `opto_light_events.csv`; the QC Viewer's `Requirement`
`Progress1`–`12`	`ParadigmState.progress`	Firmware Events: rises in Progress plus the Requirements met (`firmware_events_between`)	the ledger's and the breaking point's `FirmwareEvents`; `TrendRhoFirmware`; the light QC's `FirmwareEvents`, *firmware events disagree* and `FirmwareEventsWithoutActivity`; the light table's `FirmwareEvents`, `ProgressAtEnd`, `EventsPerActivation`
`OnMs1`–`12`	`ParadigmState.on_ms`; a missing value is 0	the light, exact: `lit()`, `lit_seconds()`; linkage without a program (identical columns); late records (above 200)	`OptoOn_sec*`, `LightOn_sec`, the light table's `LitSec` (`LitSource` = `OnMs`) and `LightOnsets`; the light QC's lit time and *broken yoking*; the light-phase summary; the timeline's light strip; the integrity report's late records

Program.txt

`opto_program.parse_program` reads three shapes behind one API, told apart by content (`OptoProgram.grammar`):

Grammar	Recognised by	What it gives
`v4`	a `Paradigms:` line, or an interval line whose head is a bare `Dark` or `Lit`	the `Paradigms:` block as `Paradigm` objects (name, one of six types, every setting); per DFM the `Linkage:`, `Program Type:` and the interval lines, thresholds a level, `On` or `Off`
`legacy`	`F:`, `P:`, `D:`, `L:`, `M:` interval lines	each line's numbers wrapped into a `Paradigm` with PyMCU's generated name (`ClosedLoop-40Hz-8ms-Hold500-MaxOn0`), `generated` set and the raw numbers in `legacy`

Grammar	Recognised by	What it gives
`simple`	a `***ALL DFM***` block	no DFM section, and the warning *recorded without a Program: every well Off*; the light QC judges every well `Off`

`***Env Mon***` is ignored. A `NAME (Type): …` line with a type not among the six is a warning, and that Paradigm is dropped.

How the program meets the rows on generation 3:

- **Row `Paradigm` → the `Paradigms:` block**: the type and settings. Only the file has them; without it every type-dependent result is `NA` and the generation-3 warning *DFM n: firmware 4.0.0 recording without Program.txt: Paradigm types and settings unknown* is raised.
- **Row `Interval` k → the DFM section's k-th printed line** (`OptoProgram.interval_for`): the thresholds of that run. The export omits Intervals whose first-pass start falls past the duration and lists a Repeating program's first pass only, so a k beyond the printed lines is judged by its Paradigm's name with thresholds unknown.
- `Linkage:` gives the Linkage Groups for the whole recording.
- **The timestamps** are used only on generations 1 and 2, to lay the intervals out by clock time; on generation 3 the rows are the schedule.

The thresholds printed are the ones PyMCU held when recording started: raw on a first run, baseline-adjusted when the same loaded program is run again without reloading. pyflic does not use them as signal levels, only to tell trigger, `On` and `Off` wells apart.

`qc/opto/opto_program.csv` (`opto_qc.OPTO_PROGRAM_COLUMNS`) is the program as read: `DFM, Interval, Start, DurationMin, ProgramType, Paradigm, ParadigmType, Generated, Dark, FrequencyHz, PulseWidthMs, HoldMs, MaxOnMs, ActivationMs, LockoutMs, OnDelayMs, EventGapMs, Acclimation, Series, FirstRequirement, Step, Scale, Rate, Cap, Ratio, Linkage, TriggerWells, OpenLoopWells, LegacyF, LegacyP`.

Messages.csv

PyMCU writes it unquoted: `DFM,Date,Time,Millisecond,Sample,Message,MsgType`, where `Sample` holds the record `Index`, `Millisecond` is an unformatted float, and the message may contain commas. `messages.read` splits on the first five commas and the last one, and returns `DFM, Time, Index, Message, MsgType`; a missing file is an empty frame. DFM 0 is the MCU itself.

- The integrity report lists the `Error` rows for the DFM and for DFM 0.
- The QC Viewer's Integrity sub-tab shows every message for the DFM and DFM 0.

What pyflic derives

Quantity	Generation 1	Generation 2	Generation 3
Training end (per well)	the last flagged minute	none: every well never flagged	the first row under a ratio Paradigm and not in Phase `A` (`training_end`); `Minutes` of the row before
A Progressive Ratio Light Event	an onset of the chamber's `OptoCol1` light	same	an Activation of the paired sucrose well
The Requirement a light event met	—	—	the row before the rise in `Activations`; on the first row of a new Paradigm, the program's `requirement_for(k)`; none in Acclimation
Firmware Events behind a light event	—	—	from the previous onset, exclusive, to this one, inclusive
Light seconds	lit `OptoCol1` samples × the sample period	same	Σ `OnMs` / 1000 (`DFM.light_seconds`, `DFM.light_source`)
Linkage without a program	wells lit identically in `OptoCol1`	same	wells whose `OnMs` columns are identical

Quantity	Generation 1	Generation 2	Generation 3
Trigger wells without a program	every lit well	same	wells with Activations
Role without a program	blank	blank	from the data
Program Facets	the program's clock schedule, approximate	same	the rows' `Interval` and `Cycle`
The light QC's contact evidence	the emulated trigger	same	Activations and Firmware Events with no pyflic activity; nothing emulated

Outputs by file

What each output gains from the new columns. Every firmware column is `NA` before generation 3.

Output	New or changed
`analysis/feeding_summary*.csv`	optogenetic members: `Role`, `LitSec`, `Activations`; `OptoOn_sec*` from `OnMs` on generation 3
`analysis/light_summary.csv`, `light_wells.csv`, their `_facet` variants	new: `light_summary.LIGHT_CHAMBER_COLUMNS` and `LIGHT_WELL_COLUMNS`
`analysis/light_phase_summary.csv`	the phase is each chamber's own light, on every generation (decision 24)
`analysis/pr_breaking_point.csv`	`BreakRequirement`, `FirmwareEvents`, `ParadigmType`; `LargestRequirement` equals `BreakRequirement` on generation 3; no row for a generation-2 group
`analysis/pr_light_events.csv`	`Requirement`, `FirmwareEvents`, `Phase`
`analysis/pr_light_qc.csv`	`TrendRhoFirmware`
`analysis/summary.txt`	each DFM's generation and light source; the Program Facets note; *no training information* for a generation-2 Progressive Ratio member
`qc/opto/opto_light_qc.csv`	`Generation`, `Roles`, `FirmwareEvents`, `PyflicEvents`, `AcclimationActivations`, `AcclimationExpected`, `FirmwareEventsWithoutActivity`; on generation 3 `ContactSec` and `ContactWithoutActivitySec` read the firmware's decisions and `EmulationApproximate` is `NA`
`qc/opto/opto_light_intervals.csv`	`ProgramInterval`, `Cycle`, `Paradigm`, `ParadigmType`, `HoldMs` (was `DecayMs`), `LitSource`
`qc/opto/opto_light_events.csv`	`Well`, `Phase`, `RequirementMet`, `FirmwareEvents`, `Source`

Output	New or changed
`qc/opto/opto_program.csv`	the columns above, on every generation; they replace `F`, `AcclimationEvents`, `P`, `NonFeeding`, `DecayMs`, `Delay` and `MaxTimeOn`
`qc/integrity/DFM<n>_integrity_report.*`	`generation`, `lost_records`, `repeated_records`, `unknown_instruction_rows`, `late_records`; the text adds the gaps, the Paradigm sequence and the `Messages.csv` errors
`qc/data_breaks/DFM<n>_data_breaks.csv`	led by `BreakIndex` and `GapSeconds` (were `Index` and `Interval`, which crashed on any version-3 file with a break)
`qc/simultaneous_feeding/DFM<n>_simultaneous_feeding_matrix.csv`	`Both_pre`, `MaxMinSignalAtBoth_pre`, `HigherInCol1AtBoth_pre`, before the dual-feeding correction
`qc/program/program_timeline_dfm<n>.png`	new
`<project>_LightSummary.csv`, `<project>_LightWells.csv` and `_Facet` variants	new: the members' light tables stacked, `Experiment` first

To confirm on the first real recording

No recording from a 4.0.0 board existed when this was built. Everything above comes from the PyMCU and firmware source and a simulator pinned to the firmware's own `LEDControl.c` (`tests/v4_fixtures.py`, cross-checked by `tests/test_ledcontrol_crosscheck.py`). When the first bench recording exists, diff it against the fixtures and the loader:

- The header is the 90 columns; every row has 90 fields; missing values are `NA`.
- The lead-in: how many `Interval` 0 rows, their `Paradigm` (`Idle` or `NA`), their `Dark`, `OptoFreq` and `OptoPW` (0).
- `Interval` changes on the first row the DFM ran the new instruction; its lag against `Program.txt`'s timestamps.
- `Activations` rises in the record where the light starts; a linked `Off` well's `OnMs` equals the trigger well's; `Requirement` during Acclimation; `Progress` is 0 on the rising row.
- `OnMs` values above 200 and how often they occur; `OptoCol1` against `OnMs > 0`.
- `Program.txt`: thresholds raw or baseline-adjusted (a re-run program), the omitted Intervals, the Repeating first pass.
- `Messages.csv`: the actual field shapes, and any message with commas.
- The five constants (`opto_event_disagreement_warn_fraction`, `opto_yoking_fail_fraction`, `opto_lost_records_warn`, `opto_unknown_rows_warn_fraction`, `opto_late_records_warn`) against what a clean recording shows.
- The simulator's signal model against real `W` traces around Activations (the 128-scan lag).

Then re-run the real Progressive Ratio dataset (generation 1) and confirm its outputs are unchanged apart from the deliberate changes in `CHANGELOG.md`.

Not read

- `Temp`, `Humid`, `LUX` and `VoltsIn`, and the Environmental Monitor's own file.
- Fine mode (firmware 4.1.0, per-scan data). `ParadigmState.index` is the key a fine table would join on.
- The firmware version: no file records it, which is why the generation is inferred.